

GCC1734 – Inteligência Artificial
Nota de Aula 03 – Busca com Informação

Prof. Eduardo Bezerra

Conteúdo

1	Revisão: Busca sem Informação	3
1.1	Limitação da UCS	3
2	Busca com Informação	4
3	Função Heurística	5
3.1	Exemplo 1: Problema de navegação (Pac-Man)	5
3.2	Exemplo 2: Problema das cidades da Romênia	6
4	Busca Gulosa (<i>Greedy Search</i>)	8
4.1	Exemplo: Romênia	8
4.2	Análise da Busca Gulosa	10
5	Busca A* (<i>A-Star Search</i>)	11
5.1	Justificativa intuitiva	12
5.2	Exemplo: grafo simples	12
5.3	Quando a Busca A* deve terminar?	14
5.4	Quando a Heurística é Ineficiente?	15
6	Busca em Árvore vs. Busca em Grafo	16
6.1	Impacto na otimalidade do A*	16
7	Otimalidade do A*	17
7.1	Heurísticas Admissíveis	17
7.2	A* com Busca em Árvore: prova de otimalidade	18
7.3	Heurísticas Consistentes	22
7.4	Relação entre admissibilidade e consistência	25
8	Propriedades do A*	26
8.1	Completeness, otimalidade e complexidade	26
8.2	Eficiência ótima	26
8.3	Comparação visual: UCS vs. A*	27

9	Construção de Heurísticas Admissíveis	27
9.1	Problemas relaxados	28
9.2	Exemplo de projeto de heurística	31
9.3	O trade-off entre qualidade da heurística e custo por nó	33
9.4	Dominância entre heurísticas	34
10	Aplicações do A*	35
11	Resumo Comparativo	36
12	Atividades Intra-Classe	36
12.1	Busca Gulosa vs. A* em um grafo simples	36
12.2	Verificação de admissibilidade e consistência	37
12.3	Construção de heurísticas por problemas relaxados	37

Objetivos

Ao final da leitura, você deve ser capaz de:

- Explicar o que é uma função heurística e por que ela é útil em problemas de busca.
- Descrever o funcionamento da Busca Gulosa (*Greedy Search*) e identificar suas limitações.
- Descrever o funcionamento da Busca A* e sua função de avaliação $f(n) = g(n) + h(n)$.
- Distinguir busca em árvore (*tree search*) de busca em grafo (*graph search*) no contexto do A*.
- Definir heurísticas admissíveis e consistentes e provar a otimalidade do A*.
- Construir heurísticas admissíveis a partir de problemas relaxados.
- Comparar o desempenho de UCS, Busca Gulosa e A* em termos de completude, otimalidade e eficiência.

1 Revisão: Busca sem Informação

Nas notas de aula anteriores, estudamos os algoritmos de busca sem informação (BFS, DFS, UCS, etc.). Nesses algoritmos, o agente explora o espaço de estados utilizando apenas a especificação do problema (estados, ações, custos e teste de objetivo) sem qualquer conhecimento sobre em que região do espaço de estados o objetivo se encontra ou quão próximo cada estado está de ser uma solução.

Todos esses algoritmos compartilham um esquema genérico: mantêm uma fronteira de nós não expandidos e selecionam um nó para expansão a cada iteração. A política de seleção é uma diferença central entre eles, mas o tratamento de estados repetidos e de caminhos de custo melhor também pode alterar suas propriedades. Em particular, a Busca de Custo Uniforme (UCS) seleciona sempre o nó de menor custo acumulado $g(n)$, o que, sob as condições usuais sobre os custos das ações, garante encontrar a solução de custo mínimo.

1.1 Limitação da UCS

Embora a UCS seja completa e ótima, ela possui uma desvantagem importante: ela explora o espaço de busca em todas as direções ao mesmo tempo, sem qualquer noção de qual direção é mais promissora para alcançar o objetivo. Isso significa que, para encontrar a solução ótima, a UCS pode expandir um número muito grande de nós que estão longe do objetivo.

Em termos visuais, se imaginarmos os nós expandidos pelo UCS como contornos ao redor do estado inicial, esses contornos crescem igualmente em todas as direções, como ondas circulares se propagando em um lago. Não há nada no algoritmo que o direcione para o objetivo. Veja a Figura 1.

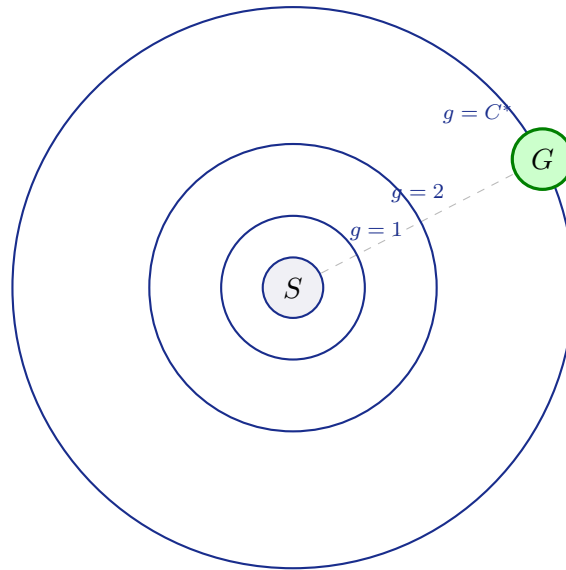


Figura 1: Expansão esquemática da UCS com custos uniformes. Os contornos de custo acumulado g crescem ao redor de S sem preferência pela direção de G ; o objetivo só é alcançado quando o contorno correspondente chega até ele.

Pergunta 1

Por que a Busca de Custo Uniforme (UCS) expande nós em todas as direções, mesmo que o objetivo esteja claramente em uma única direção? O que falta a esse algoritmo para que ele possa direcionar a busca de forma mais inteligente?

A *busca com informação* (também chamada de *busca heurística*) foi desenvolvida exatamente para superar essa limitação, incorporando ao processo de busca um conhecimento adicional sobre o problema: uma estimativa de quão próximo cada estado está do objetivo.

2 Busca com Informação

A **busca com informação** (ou *informed search*) utiliza conhecimento específico sobre o problema para encontrar soluções de forma mais eficiente do que a busca sem informação. Esse conhecimento vai além da especificação formal do problema: é um conhecimento *adicional* que o projetista do sistema fornece ao agente.

A abordagem geral é chamada de **Busca pelo Melhor Primeiro** (*Best-First Search*). Nessa abordagem:

- Uma *função de avaliação* $f(n)$ é definida para cada nó n da fronteira.
- A cada iteração, o algoritmo expande o nó com o *menor valor* de $f(n)$, ou seja, o nó considerado mais promissor.
- A escolha da função $f(n)$ determina completamente qual estratégia de busca está sendo implementada.

Do ponto de vista de implementação, algoritmos de busca pelo melhor primeiro utilizam uma fila de prioridade para representar a fronteira, com os nós ordenados pelo valor

crescente de $f(n)$. Isso é análogo ao que vimos na UCS, onde $f(n) = g(n)$. A diferença está em como $f(n)$ é definida.

Todos os algoritmos como filas de prioridade

Na prática, todos os algoritmos de busca estudados neste curso são instâncias da mesma estrutura genérica, diferindo apenas na política de ordenação da fronteira. Todos esses algoritmos podem ser vistos como instâncias de um esquema comum: a cada passo, escolhe-se da fronteira o nó com maior prioridade segundo algum critério. Em BFS, esse critério favorece nós mais rasos; em DFS, favorece o nó mais recentemente inserido; em UCS, favorece menor g ; na Busca Gulosa, menor h ; em A*, menor $g + h$.

3 Função Heurística

O elemento central da busca com informação é a *função heurística* $h(n)$.

Uma **função heurística** $h(n)$ fornece uma estimativa do custo para ir do estado representado pelo nó n até o estado objetivo mais próximo (em termos de custo).

Duas características fundamentais dessa definição merecem atenção:

- $h(n)$ é uma *estimativa*, não o custo real. O custo real para alcançar o objetivo a partir de n geralmente é desconhecido; se fosse conhecido, não seria necessário fazer busca.
- $h(n)$ é *projetada para cada problema em particular*. Não existe uma função heurística universal. Para cada domínio, o projetista deve definir uma função que capture conhecimento relevante sobre a estrutura do problema.

Formalmente, h é uma função cujo domínio é o conjunto de estados (ou nós) e cujo contradomínio é o conjunto dos reais não-negativos:

$$h : S \rightarrow \mathbb{R}_{\geq 0}$$

Para o estado objetivo s^* , temos por convenção $h(s^*) = 0$.

3.1 Exemplo 1: Problema de navegação (Pac-Man)

Considere o problema de navegar um labirinto do Pac-Man, onde o objetivo é alcançar uma pílula em alguma posição do labirinto. Suponha que agente conhece as coordenadas de sua posição atual e da pílula. Suponha também que cada ação move o agente uma unidade nas direções horizontal ou vertical e tem custo unitário. Nesse caso, duas funções heurísticas naturais são:

- **Distância Euclidiana:** a distância em linha reta entre a posição atual (x_1, y_1) e a posição da pílula (x_2, y_2) :

$$h_E(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

- **Distância de Manhattan:** a distância mínima quando são permitidos apenas movimentos horizontais e verticais:

$$h_M(n) = |x_2 - x_1| + |y_2 - y_1|$$

A Figura 2 torna explícita a diferença entre uma estimativa (gerada por $h_E(n)$ ou por $h_M(n)$) e o custo efetivo de navegação. A partir da posição (1, 1) (posição em que o agente se encontra), a distância em linha reta até a pílula em (0, 4) é $h_E = \sqrt{10}$, enquanto a distância de Manhattan é $h_M = 4$. Ambas ignoram as paredes; no labirinto, o caminho de menor custo precisa contorná-las pela direita e tem custo 8. Portanto, neste estado, $h_E \leq h_M \leq h^*(n) = 8$: as duas medidas fornecem limites inferiores úteis, mas não substituem a busca pelo caminho viável.

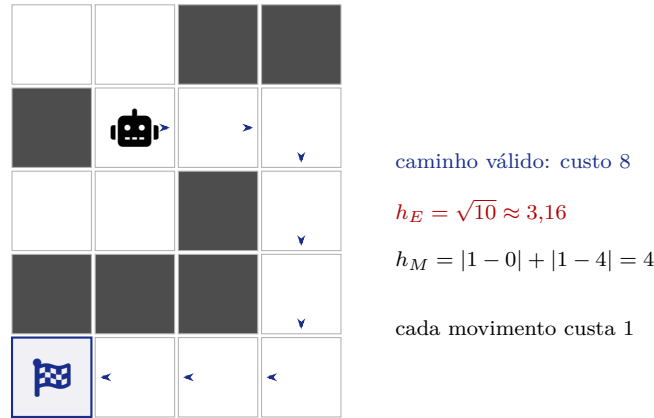


Figura 2: Labirinto do Pac-Man para ilustrar heurísticas. A distância Euclidiana e a distância de Manhattan ignoram as paredes e subestimam o custo real do caminho até a pílula. Neste estado, $h_E = \sqrt{10}$, $h_M = 4$ e o caminho válido destacado tem custo 8.

Ambas as distâncias são estimativas *otimistas*: elas nunca superestimam o custo real para chegar ao objetivo. Qualquer caminho percorrido tem comprimento pelo menos igual à distância em linha reta. Além disso, como os movimentos são apenas horizontais e verticais, até mesmo sem paredes seriam necessários pelo menos $h_M(n)$ passos; as paredes podem apenas aumentar esse custo.

Pergunta 2

Por que a distância Euclidiana entre dois pontos em um labirinto é sempre menor ou igual ao custo real para ir de um ponto ao outro? Se as paredes fossem removidas, mas os movimentos continuassem apenas horizontais e verticais com custo unitário, essa propriedade continuaria válida? O que mudaria no custo real?

3.2 Exemplo 2: Problema das cidades da Romênia

No problema das cidades da Romênia, o objetivo é encontrar a rota de menor custo entre duas cidades. A Figura 3 apresenta o mapa rodoviário utilizado neste exemplo, com as distâncias em km entre cidades vizinhas.

Uma heurística natural para estimar o custo restante até Bucareste é a *distância em linha reta* (também chamada de distância em linha de voo) desde a cidade atual

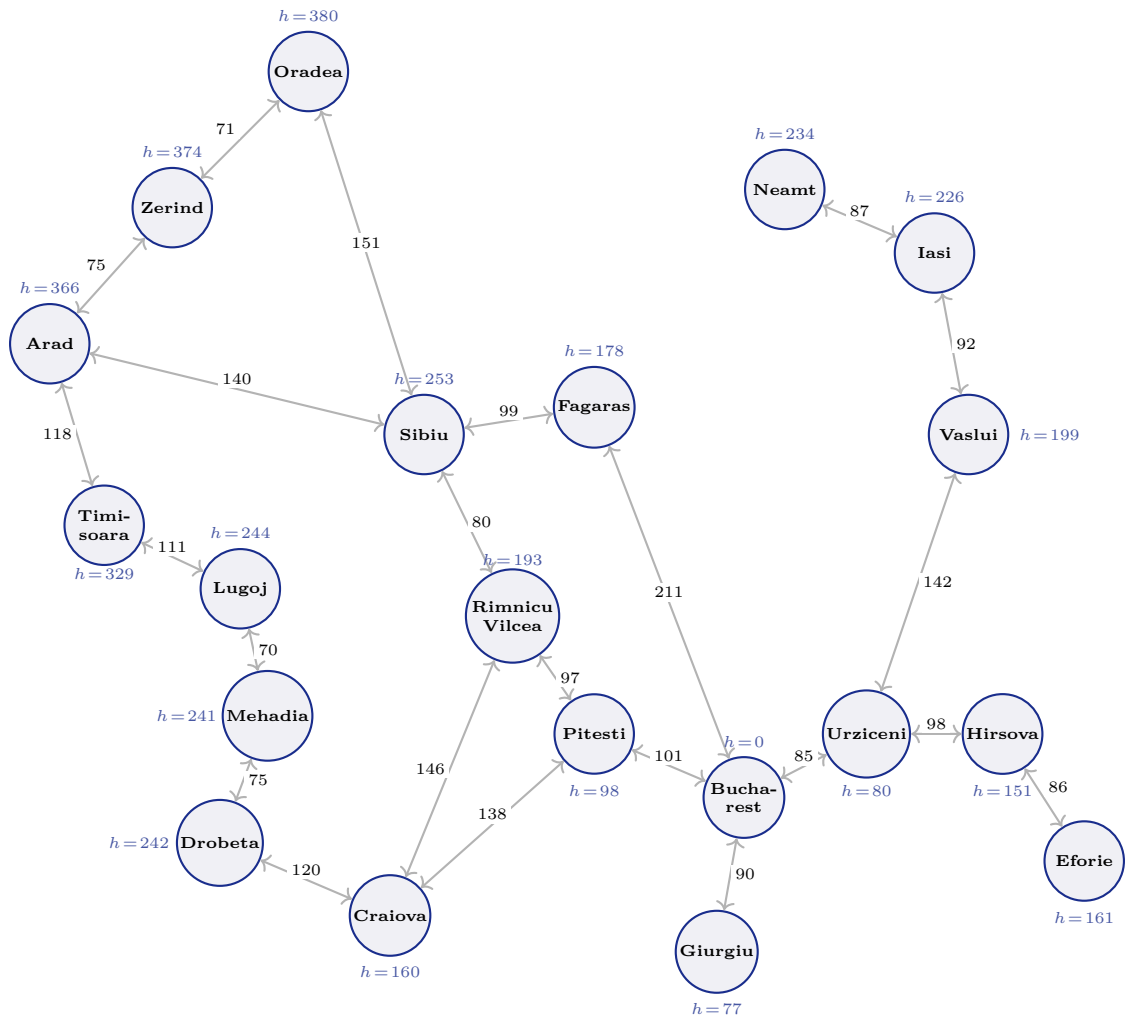


Figura 3: Mapa rodoviário simplificado da Romênia. As setas duplas indicam estradas utilizáveis nos dois sentidos; os números nas arestas indicam distâncias em km entre cidades vizinhas. Os valores $h(n)$ indicam a distância em linha reta de cada cidade até Bucareste, usada como heurística admissível nos exemplos a seguir.

até Bucareste. Esses valores estão anotados diretamente no mapa (Figura 3) e também reunidos na tabela abaixo para facilitar a consulta:

Cidade	$h(n)$ (km)
Arad	366
Bucharest	0
Craiova	160
Drobeta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Repare que, embora h seja apresentada como uma tabela, ela ainda é uma função: para cada cidade, fornece um único valor numérico. Além disso, como as estradas não são linhas retas, a distância em linha reta nunca pode superar o custo real por estrada: esta é novamente uma estimativa otimista.

4 Busca Gulosa (*Greedy Search*)

A **Busca Gulosa** (*Greedy Best-First Search*) é a estratégia mais simples de busca com informação. Ela utiliza a função de avaliação a seguir para organizar os elementos na fronteira:

$$f(n) = h(n)$$

ou seja, a prioridade de um nó é determinada exclusivamente pela estimativa heurística do custo para alcançar o objetivo a partir daquele nó. O algoritmo sempre expande o nó que parece estar mais próximo do objetivo segundo a heurística.

A intuição por trás da Busca Gulosa é simples: “se queremos chegar ao objetivo, vamos sempre em direção ao estado que parece mais próximo”. A esperança é que esse comportamento leve ao objetivo rapidamente.

4.1 Exemplo: Romênia

Considere a aplicação da Busca Gulosa ao problema de encontrar uma rota de Arad até Bucareste (Figura 3). Usaremos como heurística $h(n)$ a distância em linha reta de cada cidade até Bucareste. Como a Busca Gulosa utiliza $f(n) = h(n)$, a cidade selecionada para expansão será sempre aquela que possuir o menor valor de h entre os nós presentes

na fronteira. Observe que os custos das estradas *não participam dessa decisão*. Vamos acompanhar a execução passo a passo.

Passo 1: expansão de Arad. Inicialmente, a fronteira contém apenas Arad:

$$\text{fronteira} = \{\text{Arad } (h = 366)\}.$$

Arad é selecionada e expandida. Seus sucessores são:

Sibiu	$h = 253$
Timisoara	$h = 329$
Zerind	$h = 374$

Assim, após a expansão:

$$\text{fronteira} = \{\text{Sibiu } (253), \text{Timisoara } (329), \text{Zerind } (374)\}.$$

Como Sibiu possui o menor valor de h , ela é escolhida para a próxima expansão.

Passo 2: expansão de Sibiu. A expansão de Sibiu gera, desconsiderando Arad, que já foi expandida:

Fagaras	$h = 178$
Rimnicu Vilcea	$h = 193$
Oradea	$h = 380$

A fronteira passa a ser:

$$\begin{aligned} \text{fronteira} = \{ & \text{Fagaras } (178), \text{Rimnicu Vilcea } (193), \\ & \text{Timisoara } (329), \text{Zerind } (374), \text{Oradea } (380) \}. \end{aligned}$$

O menor valor heurístico é o de Fagaras. Portanto, a Busca Gulosa escolhe Fagaras.

Este é o passo decisivo do exemplo. Rimnicu Vilcea possui $h = 193$, enquanto Fagaras possui $h = 178$. Segundo a heurística, Fagaras *parece* estar mais próxima de Bucareste. A Busca Gulosa não considera quanto custará efetivamente seguir por cada uma dessas alternativas.

Passo 3: expansão de Fagaras. Fagaras possui uma estrada diretamente para Bucareste:

$$\text{Bucareste} : h = 0.$$

Após sua geração, a fronteira contém Bucareste com o menor valor possível de heurística:

$$\text{fronteira} = \{\text{Bucareste } (0), \text{Rimnicu Vilcea } (193), \dots\}.$$

Bucareste é então selecionada. Como ela é o estado objetivo, a busca termina. A sequência de nós selecionados foi, portanto,

$$\boxed{\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{Fagaras} \rightarrow \text{Bucareste}}$$

e o caminho encontrado possui custo

$$140 + 99 + 211 = 450 \text{ km.}$$

A solução encontrada é ótima? Não. Existe o caminho

Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu Vilcea $\rightarrow$ Pitesti $\rightarrow$ Bucareste,
cujo custo é

$$140 + 80 + 97 + 101 = 418 \text{ km.}$$

Portanto,

$$418 < 450.$$

A Busca Gulosa encontrou uma solução rapidamente, mas não encontrou a rota de menor custo.

O erro pode ser localizado na decisão tomada em Sibiu. Nesse ponto, o algoritmo comparou apenas

$$h(\text{Fagaras}) = 178 \quad \text{e} \quad h(\text{Rimnicu Vilcea}) = 193$$

e escolheu Fagaras. Contudo, o custo rodoviário restante a partir de Fagaras é

$$211 \text{ km,}$$

enquanto, seguindo por Rimnicu Vilcea e Pitesti, o custo restante é

$$97 + 101 = 198 \text{ km.}$$

Assim, a cidade que *parecia* mais próxima segundo a distância em linha reta não fazia parte do caminho rodoviário de menor custo.

O que este exemplo mostra? A Busca Gulosa toma decisões com base exclusivamente em $h(n)$. Ela não leva em consideração nem o custo já percorrido $g(n)$ nem o custo total estimado de uma solução passando por n . Por isso, uma escolha que parece melhor localmente pode levar a uma solução globalmente subótima.

Pergunta 3

Considere agora que a busca parte de Iasi com destino a Bucareste. Trace o comportamento da Busca Gulosa usando a heurística de distância em linha reta fornecida na Figura 3. A cada iteração, registre os nós relevantes da fronteira e seus valores de $h(n)$. Quais nós são selecionados até o objetivo? Qual caminho é encontrado e qual é seu custo?

4.2 Análise da Busca Gulosa

A Busca Gulosa possui as seguintes propriedades:

- **Completa?** Não. Sem detecção de estados repetidos, a Busca Gulosa pode ficar presa em ciclos de um grafo de espaço de estados. Em espaços de estados finitos, ela torna-se completa se for implementada com detecção de estados repetidos.

O próprio problema das cidades da Romênia contém ciclos, pois as estradas são bidirecionais. Por exemplo, ao expandir Arad pode-se gerar Sibiu; ao expandir

Sibiu, Arad pode ser gerada novamente. Sem controle de estados repetidos, a árvore de busca pode conter sequências como

$$\text{Arad} \rightarrow \text{Sibiu} \rightarrow \text{Arad} \rightarrow \text{Sibiu} \rightarrow \dots$$

Observe que essa sequência não significa que a heurística de distância até Bucareste necessariamente fará a Busca Gulosa seguir esse ciclo no exemplo anterior. Ela apenas mostra que o espaço de estados permite repetições. Dependendo dos valores heurísticos e da ordem de seleção da fronteira, essas repetições podem impedir a busca de alcançar o objetivo.

Em espaços de estados finitos, a Busca Gulosa torna-se completa quando implementada como busca em grafo, pois estados já expandidos não são reconsiderados.

- **Complexidade de tempo?** $O(b^m)$ no pior caso, onde b é o fator de ramificação e m é a profundidade máxima. Uma boa função heurística pode reduzir substancialmente esse tempo na prática.
- **Complexidade de espaço?** $O(b^m)$ (mantém todos os nós na memória).
- **Ótima?** Não. A Busca Gulosa escolhe ações considerando apenas a distância estimada ao objetivo a partir do estado atual, sem levar em conta o custo acumulado para chegar até esse estado. Por isso, pode existir um plano globalmente melhor que siga algumas opções localmente piores em certos pontos da árvore de busca.

A razão fundamental pela qual a Busca Gulosa não é ótima é que ela ignora o custo já pago para alcançar cada nó. Considere dois nós n_1 e n_2 na fronteira, com $h(n_1) = 5$ e $h(n_2) = 7$. A Busca Gulosa expandirá n_1 primeiro, independentemente dos custos $g(n_1)$ e $g(n_2)$. Mas se $g(n_1) = 100$ e $g(n_2) = 2$, então o caminho passando por n_2 quase certamente levará a uma solução de menor custo total, mesmo que n_2 esteja estimativamente mais distante do objetivo. A Busca Gulosa não tem como perceber isso.

Em resumo, a Busca Gulosa sacrifica tanto completude quanto otimalidade em troca de velocidade. Em muitos problemas práticos, ela encontra soluções razoavelmente boas muito rapidamente, o que a torna útil em contextos onde a qualidade exata da solução não é crítica. No pior caso, entretanto, seu comportamento pode ser similar ao de uma DFS “piorada”, visitando muitos nós sem encontrar o objetivo.

5 Busca A^* (*A-Star Search*)

A Busca Gulosa falha porque ignora o custo já pago para chegar a cada nó; ela tende a seguir a direção certa, mas pode trilhar um caminho longo para chegar lá. A UCS falha porque ignora a direção do objetivo; ela garante otimalidade, mas explora o espaço de forma uniforme, sem preferência pela direção certa. A **Busca A^*** (pronuncia-se “A-estrela”) supera ambos os problemas ao combinar os dois critérios em uma única função de avaliação:

$$f(n) = g(n) + h(n)$$

onde:

- $g(n)$ é o custo acumulado desde o estado inicial até o nó n (custo *para trás*, já conhecido);
- $h(n)$ é a estimativa heurística do custo desde n até o objetivo (custo *para frente*, estimado);
- $f(n)$ é, portanto, a estimativa do custo total do caminho que passa por n .

5.1 Justificativa intuitiva

A justificativa do A^* é direta: se queremos encontrar a solução de menor custo total, devemos expandir o nó que representa o caminho com a menor estimativa de custo total. O valor $f(n) = g(n) + h(n)$ é exatamente essa estimativa: o custo já pago mais o custo estimado restante.

Comparando as três estratégias:

- **UCS:** ordena a fronteira pelo custo de caminho $g(n)$. É ótima, mas ignora a direção do objetivo; expande nós em todas as direções igualmente.
- **Busca Gulosa:** ordena pela estimativa $h(n)$. Tende a ir na direção certa, mas ignora o custo já pago; pode chegar ao destino por um caminho muito longo.
- **A^* :** ordena pela soma $f(n) = g(n) + h(n)$. Combina os dois critérios: dirige-se para o objetivo sem ignorar o custo acumulado.

A UCS é um caso especial do A^* com $h(n) = 0$ para todo n .

Pergunta 4

A Busca Gulosa utiliza apenas o termo $h(n)$ da função de avaliação do A^* , ignorando $g(n)$. Que consequência isso tem para a noção de “custo pago até agora”? Por que ignorar $g(n)$ compromete a otimalidade?

Pergunta 5

Explique, com suas próprias palavras, por que usar apenas $g(n)$ (como a UCS) pode levar a expandir muitos nós desnecessários, e por que usar apenas $h(n)$ (como a Busca Gulosa) pode levar a um plano subótimo. Como a combinação $f(n) = g(n) + h(n)$ tenta resolver ambos os problemas simultaneamente?

5.2 Exemplo: grafo simples

Para ilustrar as diferenças entre UCS, Busca Gulosa e A^* , considere o grafo da Figura 4. O estado inicial é S , o objetivo é G , os números associados às arestas indicam os custos das ações, e os valores junto a cada nó indicam a heurística $h(n)$.

A Figura 4 foi construída para destacar uma situação em que o custo acumulado e a estimativa fornecida pela heurística fornecem informações diferentes. O nó d funciona como uma espécie de *armadilha*: ele é alcançado com baixo custo a partir de S , pois $g(d) = 2$, mas está longe do objetivo, já que o custo real de d até G é 10. Em contraste, o caminho superior

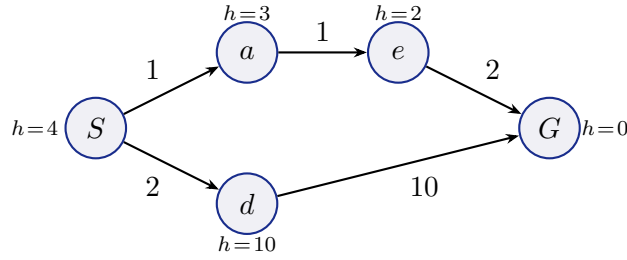


Figura 4: Grafo usado para comparar UCS, Busca Gulosa e A*. O caminho superior $S \rightarrow a \rightarrow e \rightarrow G$ tem custo total 4 e é ótimo. O nó d é barato de alcançar a partir de S , mas está associado a um caminho caro até o objetivo, o que permite evidenciar as diferenças entre os critérios $g(n)$, $h(n)$ e $g(n) + h(n)$.

$$S \rightarrow a \rightarrow e \rightarrow G$$

possui custo total

$$1 + 1 + 2 = 4,$$

sendo, portanto, o caminho ótimo, com $C^* = 4$.

Esse mesmo grafo será usado a seguir para mostrar como UCS, Busca Gulosa e A* ordenam a fronteira de maneiras diferentes e, conseqüentemente, podem expandir conjuntos distintos de nós.

Busca de Custo Uniforme (UCS) (ordena os nós pelo custo acumulado g).

- Expande S ($g = 0$) e gera a ($g = 1$) e d ($g = 2$).
- Expande a ($g = 1$) e gera e ($g = 2$).
- Fronteira: $\{d$ ($g = 2$), e ($g = 2$) $\}$. Há empate; suponha que d é expandido primeiro (escolhemos este desempate propositalmente para mostrar a limitação da UCS).
- **Expande** d ($g = 2$) e gera G ($g = 12$).
- Expande e ($g = 2$) e gera G ($g = 4$) (caminho mais barato).
- Expande G ($g = 4$) e termina.

A UCS encontra o caminho ótimo, mas expandiu d desnecessariamente. Sem qualquer noção de distância ao objetivo, ela não distingue d (que está perto de S porém longe de G) de e (que está igualmente perto de S e também perto de G).

Busca Gulosa (ordena os nós apenas pela heurística h).

- Expande S ($h = 4$) e gera a ($h = 3$) e d ($h = 10$).
- Expande a ($h = 3$) e gera e ($h = 2$).
- Expande e ($h = 2$) e gera G ($h = 0$).
- Expande G e termina.

A Busca Gulosa encontra o caminho ótimo e não expande d . No entanto, isso ocorreu porque a heurística, neste exemplo específico, não induziu o algoritmo a um caminho subótimo, o que não é garantido em geral: a heurística apontou diretamente para o objetivo. Em geral, a Busca Gulosa não garante otimalidade, pois ignora o custo acumulado g .

Busca A* (ordena os nós pelo valor $f(n) = g(n) + h(n)$).

- Expande S e gera:

$$a : f = 1 + 3 = 4 \quad d : f = 2 + 10 = 12$$

- Expande a (menor $f = 4$) e gera:

$$e : f = 2 + 2 = 4$$

- Expande e (menor $f = 4$) e gera:

$$G : f = 4 + 0 = 4$$

- Expande G e termina. O nó d nunca foi expandido.

O A* encontra o caminho ótimo $S \rightarrow a \rightarrow e \rightarrow G$ com custo 4 sem expandir d . A heurística $h(d) = 10$ sinalizou que d está longe do objetivo, elevando $f(d) = 12 > C^* = 4$ e mantendo d no fundo da fila de prioridade durante toda a execução.

A tabela abaixo resume o comportamento dos três algoritmos neste exemplo.

Algoritmo	Nós expandidos	Expande d ?	Custo da solução
UCS	S, a, d, e, G	Sim	4 (ótimo)
Busca Gulosa	S, a, e, G	Não	4 (ótimo neste caso)
A*	S, a, e, G	Não	4 (ótimo)

A UCS e o A* garantem a solução ótima, mas a UCS desperdiça esforço expandindo d . O A* evita esse desperdício porque combina o custo real g com a estimativa heurística h : mesmo que d seja barato de alcançar, seu f elevado revela que o caminho total passando por ele será caro.

5.3 Quando a Busca A* deve terminar?

Um detalhe importante: A* deve encerrar somente quando um nó objetivo é *retirado da fronteira para seleção*, e não quando um nó objetivo é *inserido* na fronteira.

O motivo é que, no momento em que um nó objetivo é gerado, pode ainda existir na fronteira outro caminho até o objetivo (ou até o mesmo nó objetivo) com custo menor. Ao aguardar até a remoção, garantimos que o nó objetivo expandido tem o menor f dentre todos os nós na fronteira, o que (sob certas condições sobre h) implica que seu custo g é o mínimo possível.

Como exemplo, considere o grafo apresentado na Figura 5. O A* expande S e gera dois nós: A ($f = 1 + 1 = 2$) e G via S ($f = 5 + 0 = 5$). Se o algoritmo parasse no momento em que G é *gerado* (inserido na fronteira pela aresta $S \rightarrow G$), retornaria o caminho $S \rightarrow G$ com custo 5. No entanto, A ainda está na fronteira com $f = 2 < 5$, e ao

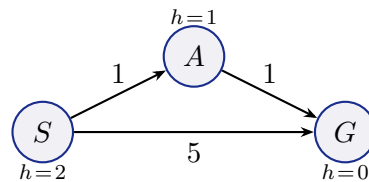


Figura 5: Exemplo em que o A* não deve terminar quando o objetivo é apenas gerado. Embora G seja alcançado diretamente a partir de S com custo 5, o nó A permanece na fronteira com menor valor de f e leva posteriormente a uma solução de custo 2.

expandi-lo geramos G com $g = 2$ (o caminho $S \rightarrow A \rightarrow G$ custa apenas 2). A informação que faltava no momento da inserção antecipada era a existência de um caminho mais barato que ainda não havia sido descoberto.

Pergunta 6

Por que o critério de parada “encerrar ao gerar o objetivo” pode retornar uma solução subótima? Usando o exemplo acima, explique por que aguardar a seleção do objetivo na fronteira evita esse erro.

5.4 Quando a Heurística é Ineficiente?

É um erro comum pensar que qualquer heurística, por mais simples que seja, sempre melhorará o desempenho computacional da busca. Uma heurística pode ser pouco informada ou custosa de calcular, sem necessariamente reduzir o esforço total de forma compensadora.

- **Heurísticas Inúteis:** Se a função $h(n)$ retorna um valor constante ($h(n) = c$ para todo n), ela não fornece nenhuma preferência real. A busca torna-se, na prática, uma busca de custo uniforme (UCS), mas com a sobrecarga computacional de calcular o valor constante a cada passo.
- **Heurísticas Pouco Informadas:** Uma heurística pode ser admissível e, portanto, preservar a otimalidade, mas não contribuir para reduzir o espaço de estados de forma significativa. Em casos extremos, se a heurística for muito imprecisa, o algoritmo A* pode expandir um número de nós muito próximo ao da UCS, porém com o custo adicional de calcular a própria função heurística.
- **Heurísticas Inadmissíveis:** Uma heurística que superestima custos pode alterar a ordem de expansão de modo a retornar uma solução subótima. Mesmo quando a qualidade da solução não é crítica, o custo de calcular uma heurística ou uma ordenação pouco útil pode tornar a execução mais lenta que uma busca cega.

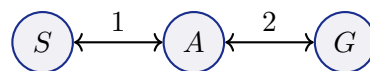
A lição fundamental é: a qualidade de uma heurística é medida pela sua capacidade de reduzir o *fator de ramificação efetivo* da busca. No entanto, o desempenho real do sistema também depende da viabilidade computacional: se o tempo gasto para calcular $h(n)$ em cada nó for superior ao tempo economizado pela redução do número de nós expandidos, a heurística, embora teoricamente boa, torna-se ineficiente na prática.

6 Busca em Árvore vs. Busca em Grafo

Antes de analisar as condições de otimalidade do A^* , é necessário recordar a distinção entre as duas formas de implementar qualquer estratégia de busca: a **busca em árvore** (*tree search*) e a **busca em grafo** (*graph search*).

- **Busca em árvore.** Na busca em árvore, o algoritmo expande nós sem verificar se o estado correspondente já foi visitado anteriormente. Isso significa que o mesmo estado pode aparecer múltiplas vezes na árvore de busca, em profundidades diferentes, correspondendo a diferentes caminhos desde o estado inicial.

A vantagem da busca em árvore é sua simplicidade: não é necessário manter um conjunto de estados explorados. A desvantagem é que, na presença de ciclos no grafo de espaço de estados, a árvore de busca pode crescer infinitamente. Por exemplo, no grafo abaixo, ela contém os caminhos cíclicos $S \rightarrow A \rightarrow S \rightarrow A \rightarrow \dots$. Dependendo da estratégia de expansão e da posição do objetivo, uma busca sem detecção de ciclos pode deixar de terminar, embora outra estratégia possa alcançar G antes de explorar indefinidamente o ciclo:



- **Busca em grafo.** Na busca em grafo, o algoritmo mantém um *conjunto explorado* (*closed set* na literatura anglófona). Antes de expandir um nó, verifica-se se seu estado já foi expandido anteriormente. Em caso positivo, o nó é ignorado.

A ideia central é *nunca expandir o mesmo estado mais de uma vez*. Isso garante terminação em espaços de estados finitos e reduz o trabalho total de exploração.

Cuidado com a nomenclatura! Tanto a busca em grafo quanto a busca em árvore *produzem* uma árvore de busca durante a execução. O que muda não é a estrutura produzida, mas sim se estados repetidos são ou não detectados. Além disso, o espaço de estados é sempre representado como um grafo (mesmo que seja uma árvore, sem ciclos). A nomenclatura “busca em árvore” e “busca em grafo” refere-se ao algoritmo de exploração, não à estrutura do problema.

6.1 Impacto na otimalidade do A^*

A distinção entre busca em árvore e busca em grafo é crucial para entender as condições de otimalidade do A^* . A garantia de otimalidade do A^* está relacionada a duas propriedades aplicáveis a heurísticas: *admissibilidade* e *consistência*.

- Para a *busca em árvore*, o A^* é ótimo se a heurística for *admissível*.
- Para a *busca em grafo*, o A^* é ótimo se a heurística for *consistente* (uma condição mais forte).

A razão para essa diferença, assim como as definições de admissibilidade e de consistência são apresentadas nas seções a seguir.

7 Otimalidade do A*

7.1 Heurísticas Admissíveis

Uma heurística $h(n)$ é **admissível** (ou *otimista*) se ela nunca superestima o custo real para alcançar o objetivo a partir de n . Formalmente:

$$h(n) \leq h^*(n) \quad \text{para todo } n$$

onde $h^*(n)$ é o custo real ótimo desde n até o objetivo mais próximo.

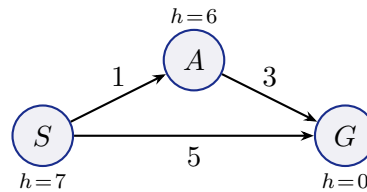
Uma heurística admissível é *otimista* no sentido de que ela sempre acredita que o custo restante é menor ou igual ao que realmente é. Isso pode ser pensado como: o agente com heurística admissível sempre vê o copo “meio cheio”: ele nunca superestima a dificuldade do que falta fazer.

Heurísticas *inadmissíveis* (pessimistas) comprometem a otimalidade porque “prendem” bons planos na fronteira: ao superestimar o custo de certos caminhos, o algoritmo pode preferir planos que, na verdade, levam a soluções ótimas.

Exemplo 1: heurística admissível Os valores heurísticos mostrados na Figura 4 são admissíveis, pois nunca superestimam o custo real restante até G :

$$h(S) = 4 \leq 4, \quad h(a) = 3 \leq 3, \quad h(e) = 2 \leq 2, \quad h(d) = 10 \leq 10, \quad h(G) = 0.$$

Exemplo 2: heurística inadmissível Considere o seguinte grafo simples:



Vamos verificar a admissibilidade da heurística comparando $h(n)$ com o custo real ótimo $h^*(n)$ para cada estado:

$$h^*(S) = \min(5, 1 + 3) = 4, \quad h^*(A) = 3, \quad h^*(G) = 0.$$

Portanto:

$$h(S) = 7 > 4 = h^*(S), \quad h(A) = 6 > 3 = h^*(A), \quad h(G) = 0 = h^*(G).$$

A heurística é inadmissível em **dois** estados: tanto em S quanto em A , ela superestima o custo real até o objetivo.

Agora vamos executar A* para verificar qual o plano encontrado neste caso. Executando o A*, o algoritmo expande S e insere na fronteira:

$$A : f = 1 + 6 = 7, \quad G : f = 5 + 0 = 5.$$

O nó G (alcançado diretamente por S) tem menor f e é selecionado para expansão. O algoritmo retorna o caminho $S \rightarrow G$ com custo 5.

No entanto, o caminho ótimo é $S \rightarrow A \rightarrow G$, com custo $1 + 3 = 4$. A heurística pessimista em A elevou artificialmente o valor $f(A) = 7$, fazendo o algoritmo preferir um caminho que, na realidade, é mais barato. Note que a inadmissibilidade em S reforça o mesmo efeito: a estimativa exagerada do custo restante já a partir do estado inicial contribui para que o caminho direto $S \rightarrow G$ pareça mais atraente do que realmente é.

Exemplo 3: 8-puzzle. Para o quebra-cabeça de 8 peças, duas heurísticas admissíveis são clássicas:

- $h_1(n)$: número de peças fora de sua posição no objetivo. Esta heurística é admissível porque cada peça fora do lugar precisa de pelo menos um movimento para atingir sua posição correta. Logo, $h_1(n) \leq h^*(n)$.
- $h_2(n)$: soma das distâncias de Manhattan de cada peça até sua posição no objetivo. Esta heurística é admissível. Cada movimento altera a posição de uma única peça em uma unidade horizontal ou vertical. Portanto, para levar cada peça à sua posição objetivo, são necessários pelo menos tantos movimentos quanto sua distância de Manhattan. Somando sobre todas as peças, obtemos um limite inferior para o custo ótimo.

Em geral, $h_2(n) \geq h_1(n)$ para todo n , e portanto h_2 é uma heurística *mais informada* que h_1 .

Pergunta 7

Para o estado inicial do quebra-cabeça de 8 peças abaixo, calcule h_1 e h_2 :

7	2	4
5		6
8	3	1

Estado inicial

	1	2
3	4	5
6	7	8

Estado objetivo

Ambas as heurísticas são admissíveis? Justifique.

7.2 A* com Busca em Árvore: prova de otimalidade

Teorema

Se $h(n)$ é admissível, então A* usando busca em árvore (*tree search*) é ótimo.

Vamos desenvolver a seguir uma prova desse teorema. A ideia central é comparar dois tipos de nós que podem estar na fronteira: um nó pertencente a um caminho ótimo e um nó objetivo correspondente a uma solução subótima.

Seja C^* o custo de uma solução ótima. Mostraremos que, enquanto a solução ótima ainda não tiver sido selecionada, existe na fronteira algum nó n de um caminho ótimo com

$$f(n) \leq C^*,$$

enquanto qualquer objetivo subótimo B satisfaz

$$f(B) > C^*.$$

Como o A^* sempre seleciona o nó de menor f , essas duas desigualdades serão suficientes para estabelecer a otimalidade.

Passo 1: notação. Considere um nó objetivo A correspondente a uma solução ótima. Como A é objetivo,

$$h(A) = 0,$$

e, portanto,

$$f(A) = g(A) = C^*.$$

Considere agora outro nó objetivo B , correspondente a uma solução subótima. Nesse caso,

$$g(B) > C^*.$$

Como B também é objetivo,

$$h(B) = 0,$$

logo

$$f(B) = g(B) > C^*.$$

Portanto, qualquer objetivo subótimo possui valor de f estritamente maior que C^* .

Passo 2: sempre existe na fronteira um nó de um caminho ótimo. Considere um caminho ótimo desde o estado inicial até A :

$$s_0 \rightarrow n_1 \rightarrow n_2 \rightarrow \cdots \rightarrow n_k \rightarrow A.$$

Enquanto A ainda não tiver sido selecionado, existe na fronteira pelo menos um nó desse caminho.

Para entender por quê, considere inicialmente s_0 . Antes da primeira expansão, ele próprio está na fronteira e pertence ao caminho ótimo.

Quando um nó desse caminho é selecionado e expandido, seu sucessor no mesmo caminho é gerado e inserido na fronteira. Portanto, a expansão de um nó do caminho ótimo apenas faz avançar, em uma posição, a “ponta” desse caminho que permanece na fronteira.

Assim, enquanto A ainda não tiver sido selecionado, algum nó do caminho ótimo permanece na fronteira. Denotemos esse nó por n .

A Figura 6 representa esquematicamente essa situação.

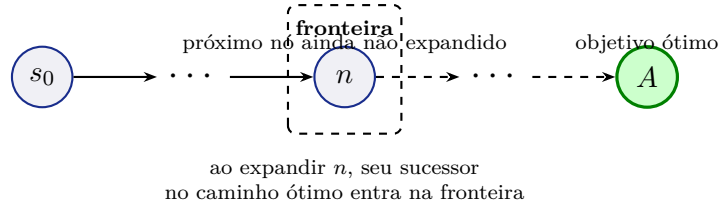


Figura 6: Existência de um nó do caminho ótimo na fronteira. Enquanto o objetivo ótimo A ainda não tiver sido selecionado, pelo menos um nó n pertencente a um caminho ótimo permanece na fronteira. Quando esse nó é expandido, seu sucessor no mesmo caminho é gerado.

Passo 3: quanto pode valer $f(n)$ nesse caminho ótimo? Como n pertence a um caminho ótimo, o custo já percorrido até ele é $g(n)$ e o custo ótimo restante até o objetivo é $h^*(n)$.

Como a heurística é admissível,

$$h(n) \leq h^*(n).$$

Somando $g(n)$ aos dois lados,

$$g(n) + h(n) \leq g(n) + h^*(n).$$

Como $f(n) = g(n) + h(n)$,

$$f(n) \leq g(n) + h^*(n).$$

Mas n está em um caminho ótimo. Portanto, o custo já percorrido até n mais o custo ótimo restante a partir de n é exatamente o custo da solução ótima:

$$g(n) + h^*(n) = C^*.$$

Logo,

$$\boxed{f(n) \leq C^*}.$$

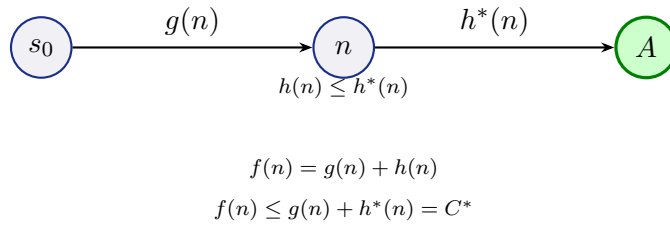


Figura 7: Limite superior para $f(n)$ em um caminho ótimo. Pela admissibilidade, $h(n) \leq h^*(n)$. Como n pertence a um caminho ótimo, $g(n) + h^*(n) = C^*$. Portanto, $f(n) = g(n) + h(n) \leq C^*$.

A Figura 7 resume visualmente a desigualdade central do Passo 3. Em particular, ela destaca que, para um nó n pertencente a um caminho ótimo, vale

$$f(n) \leq C^*.$$

Esse é o ponto central da prova: enquanto a solução ótima ainda não tiver sido selecionada, existe na fronteira pelo menos um nó n de um caminho ótimo com

$$f(n) \leq C^*.$$

Passo 4: comparação com um objetivo subótimo. Considere agora um momento da busca em que um objetivo subótimo B esteja na fronteira. Pelo Passo 1,

$$f(B) = g(B) > C^*.$$

Enquanto o objetivo ótimo A ainda não tiver sido selecionado, o Passo 2 garante que existe na fronteira algum nó n de um caminho ótimo. Pelo Passo 3,

$$f(n) \leq C^*.$$

Portanto,

$$f(n) \leq C^* < f(B),$$

e, conseqüentemente,

$$f(n) < f(B).$$

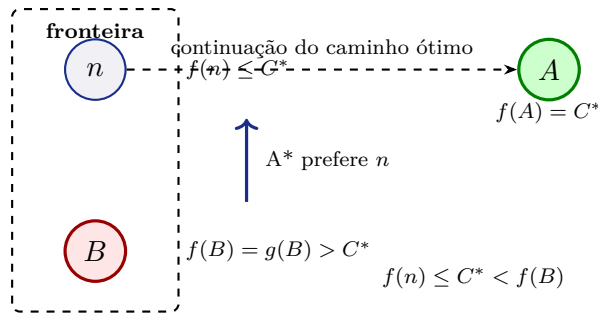


Figura 8: Comparação no momento em que um objetivo subótimo B está na fronteira. Enquanto o objetivo ótimo ainda não tiver sido selecionado, há também na fronteira algum nó n de um caminho ótimo, com $f(n) \leq C^*$. Como $f(B) = g(B) > C^*$, o A^* seleciona n antes de B .

A Figura 8 resume a comparação decisiva da prova:

$$f(n) \leq C^* < f(B).$$

Como o A^* sempre seleciona o nó da fronteira com menor valor de f , ele necessariamente selecionará n antes de B .

Depois que n for expandido, o próximo nó do caminho ótimo será inserido na fronteira. O mesmo argumento poderá então ser aplicado novamente, até que o próprio objetivo ótimo A seja selecionado.

Conclusão. Enquanto o objetivo ótimo A ainda não tiver sido selecionado, o Passo 2 garante a existência, na fronteira, de algum nó n pertencente a um caminho ótimo. Pelo Passo 3,

$$f(n) \leq C^*.$$

Por outro lado, pelo Passo 1, qualquer objetivo subótimo B satisfaz

$$f(B) > C^*.$$

Logo,

$$f(n) < f(B).$$

Portanto, sempre que um objetivo subótimo estiver na fronteira, haverá um nó de um caminho ótimo com prioridade maior para o A^* . Um objetivo subótimo não pode, assim, ser o primeiro objetivo selecionado.

Consequentemente, o primeiro nó objetivo selecionado pelo A^* corresponde a uma solução de custo C^* .

Assim, A^* com busca em árvore e heurística admissível é ótimo. $\square$

Pergunta 8

Na prova acima, por que é essencial que a heurística seja admissível? Em que passo da demonstração o argumento deixaria de funcionar se fosse possível ter $h(n) > h^*(n)$?

7.3 Heurísticas Consistentes

Para a busca em grafo, a admissibilidade não é suficiente para garantir a otimalidade do A^* . Precisamos de uma condição mais forte: a *consistência* (também chamada de *monotonicidade*).

Uma heurística $h(n)$ é *consistente* (ou *monotônica*) se, para todo nó n e todo sucessor n' de n gerado pela ação a :

$$h(n) \leq c(n, a, n') + h(n')$$

onde $c(n, a, n')$ é o custo da ação a que leva de n a n' .

Essa condição é conhecida como *desigualdade triangular*: ela é análoga à propriedade geométrica segundo a qual o comprimento de um lado de um triângulo não excede a soma dos outros dois. Aqui, a estimativa direta de n ao objetivo não pode ser maior que o custo real para chegar a n' somado à estimativa restante a partir de n' .

Assim, os dois lados que passam por n' formam um limite para o lado direto estimado por $h(n)$. A analogia é estrutural, e não geométrica: os estados não precisam ocupar posições em um espaço físico.

Essa desigualdade tem uma consequência decisiva para o A^* . Como será demonstrado formalmente adiante, se h é consistente, o valor $f(n) = g(n) + h(n)$ não diminui ao longo de um caminho. Portanto, quando um estado é expandido pela primeira vez na busca em

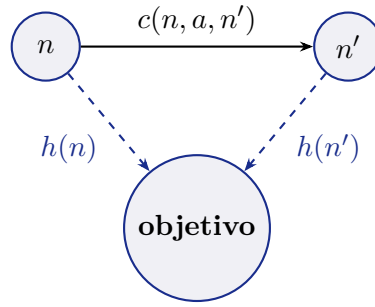


Figura 9: Interpretação esquemática da consistência. A aresta sólida é uma ação real; os segmentos tracejados representam estimativas heurísticas, não ações disponíveis. A consistência exige $h(n) \leq c(n, a, n') + h(n')$.

grafo, o caminho que o alcançou já tem custo mínimo. Sem consistência, essa propriedade pode falhar: a busca pode fechar um estado cedo demais e descartar um caminho melhor descoberto posteriormente.

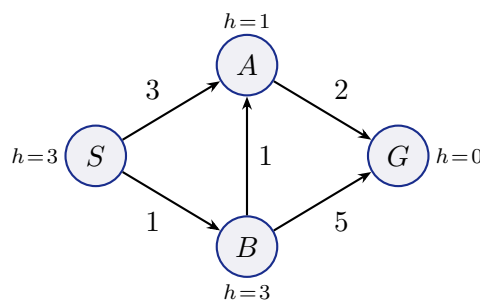
Por que a admissibilidade não basta na busca em grafo?

Na busca em grafo, o algoritmo mantém um conjunto de estados já expandidos (*closed set*). Quando um estado é expandido, ele é colocado nesse conjunto e, na versão mais simples do algoritmo, nunca mais é reconsiderado.

Isso é seguro quando a heurística é *consistente*. Nesse caso, quando um estado é selecionado para expansão pela primeira vez, o caminho que o alcançou possui custo mínimo.

Se a heurística for apenas *admissível*, porém *não consistente*, essa propriedade deixa de valer: o primeiro caminho encontrado até um estado pode não ser o melhor. Em consequência, a busca em grafo pode fechar um estado cedo demais e impedir que um caminho melhor até esse mesmo estado seja explorado posteriormente.

O exemplo a seguir ilustra essa situação.



Custos reais ótimos até o objetivo.

$$h^*(A) = 2, \quad h^*(B) = 3, \quad h^*(S) = 4$$

Logo,

$$h(A) = 1 \leq 2, \quad h(B) = 3 \leq 3, \quad h(S) = 3 \leq 4$$

Portanto, a heurística é admissível.

Verificação de consistência.

Para a aresta $B \rightarrow A$, a consistência exigiria:

$$h(B) \leq c(B, A) + h(A)$$

isto é,

$$3 \leq 1 + 1 = 2,$$

o que é falso. Logo, a heurística não é consistente.

Execução do A* com busca em grafo.

1. Expande S e gera:

$$A : g = 3, f = 3 + 1 = 4, \quad B : g = 1, f = 1 + 3 = 4$$

2. Há empate. Suponha que A seja selecionado para expansão primeiro. A expansão de A gera:

$$G : g = 5$$

Agora a fronteira contém B e G .

3. Em seguida o algoritmo expande B . A partir de B é encontrado um caminho melhor para A :

$$g(A) = 1 + 1 = 2$$

Esse custo é menor que o custo anterior ($g = 3$) obtido diretamente a partir de S .

No entanto, o estado A já está no conjunto explorado e não será reconsiderado. Assim, o algoritmo ignora um caminho melhor porque o estado foi fechado anteriormente.

4. O algoritmo então expande G e retorna a solução

$$S \rightarrow A \rightarrow G$$

com custo total 5.

No entanto, existe um plano melhor:

$$S \rightarrow B \rightarrow A \rightarrow G$$

com custo total

$$1 + 1 + 2 = 4.$$

Portanto, o A* com busca em grafo retorna uma solução **subótima** neste exemplo, mesmo usando uma heurística admissível.

Consequências da consistência

Teorema

Se h é consistente, então $f(n) = g(n) + h(n)$ é não-decrescente ao longo de qualquer caminho na árvore de busca.

Prova. Sejam n e n' dois nós consecutivos na árvore, com n' sendo filho de n pela ação a :

$$\begin{aligned} f(n') &= g(n') + h(n') \\ &= g(n) + c(n, a, n') + h(n') \\ &\geq g(n) + h(n) \quad (\text{pela consistência: } c(n, a, n') + h(n') \geq h(n)) \\ &= f(n) \end{aligned}$$

Logo, $f(n') \geq f(n)$. $\square$

Essa propriedade tem uma consequência prática importante: se a heurística é consistente, os nós são expandidos em ordem não-decrescente de f . Isso significa que, quando um estado é expandido pela primeira vez, o caminho que levou até ele é necessariamente o de menor custo. Por isso, a busca em grafo nunca precisa “corrigir” um estado já inserido no conjunto explorado.

Teorema

Consistência implica admissibilidade.

Esboço da prova. A prova é feita por indução no número de passos até o objetivo. Se n é o próprio objetivo, $h(n) = 0 = h^*(n)$. Caso contrário, seja n' o próximo estado no caminho ótimo de n ao objetivo. Pela consistência, $h(n) \leq c(n, a, n') + h(n')$. Por hipótese de indução, $h(n') \leq h^*(n')$. Como $c(n, a, n') + h^*(n') = h^*(n)$ (o passo ótimo mais o restante ótimo é o caminho ótimo total), conclui-se $h(n) \leq h^*(n)$. $\square$

Na prática, quase todas as heurísticas admissíveis naturais também são consistentes, especialmente aquelas derivadas de problemas relaxados (como veremos na Seção 9).

Teorema

Se $h(n)$ é consistente, então A^* usando busca em grafo (*graph search*) é ótimo.

O esboço da prova baseia-se em dois fatos decorrentes da consistência:

1. A^* expande nós em ordem não-decrescente de f (provado acima).
2. Para todo estado s , o nó que alcança s otimamente (com o menor g) é sempre expandido antes do nó que alcança s de forma subótima.

Desses dois fatos segue que, quando o objetivo é expandido, o custo acumulado g é o menor possível, e portanto a solução é ótima.

7.4 Relação entre admissibilidade e consistência

A tabela abaixo resume os requisitos de otimalidade para o A^* :

Modo de busca	Condição sobre h	A^* é ótimo?
Busca em árvore (<i>tree search</i>)	Admissível	Sim
Busca em grafo (<i>graph search</i>)	Consistente	Sim
Busca em grafo (<i>graph search</i>)	Apenas admissível	Não necessariamente

Pergunta 9

Por que a admissibilidade não é suficiente para garantir a otimalidade do A^* com busca em grafo? Qual é o papel do conjunto explorado nessa falha? O que a consistência garante que resolve o problema?

8 Propriedades do A^*

8.1 Completude, otimalidade e complexidade

- **Completo?** Sim, desde que o fator de ramificação seja finito e os custos das ações sejam estritamente positivos (limitados inferiormente por algum $\varepsilon > 0$). Sem essa condição, pode existir uma quantidade infinita de caminhos de custo menor do que o da solução ótima, impedindo a terminação.
- **Complexidade de tempo?** Exponencial no pior caso: $O(b^{\lceil C^*/\varepsilon \rceil})$, onde b é o fator de ramificação, C^* é o custo da solução ótima e $\varepsilon > 0$ é um limite inferior para o custo de cada ação. Com custos unitários, essa expressão se reduz a $O(b^d)$, onde d é a profundidade da solução ótima. Na prática, uma boa heurística pode reduzir drasticamente o número de nós expandidos.
- **Complexidade de espaço?** Exponencial também: o A^* mantém todos os nós gerados na memória (tanto a fronteira quanto o conjunto explorado), o que pode ser proibitivo para problemas grandes.
- **Ótimo?** Sim, com heurística admissível (busca em árvore) ou consistente (busca em grafo).

8.2 Eficiência ótima

O A^* possui uma propriedade notável além da otimalidade:

Teorema: eficiência ótima do A^*

Em busca em árvore, com heurística admissível, nenhum algoritmo garantidamente ótimo que use apenas a mesma informação pode evitar, em todos os problemas, a expansão de um nó com $f(n) < C^*$, onde C^* é o custo da solução ótima. Nós com $f(n) = C^*$ constituem uma exceção: sua expansão pode depender da regra de desempate.

Isso significa que o A^* é otimamente eficiente dentro dessas condições. Para reduzir de forma garantida o número de expansões, é necessário acrescentar informação ao problema, por exemplo usando uma heurística mais informada.

8.3 Comparação visual: UCS vs. A*

Uma forma geométrica de comparar UCS e A* é considerar curvas de nível das funções utilizadas para ordenar a fronteira.

- A **UCS** ordena os nós apenas pelo custo acumulado $g(n)$. Em um espaço idealizado com custo proporcional à distância, curvas de mesmo g são circunferências centradas em S .
- O **A*** ordena os nós por $f(n) = g(n) + h(n)$. Se $g(n)$ for a distância desde S e $h(n)$ a distância até G , curvas de mesmo f são elipses cujos focos são S e G . A informação heurística faz, portanto, com que a prioridade dependa simultaneamente da distância já percorrida e da distância estimada restante.

A Figura 10 ilustra essa diferença de forma geométrica. No painel da esquerda, a UCS considera apenas $g(n)$, produzindo curvas de mesmo custo acumulado centradas em S . No painel da direita, o A* considera $f(n) = g(n) + h(n)$; no caso idealizado em que $g(n)$ e $h(n)$ são distâncias, as curvas de mesmo valor de f são elipses com focos em S e G . A figura mostra, portanto, como a heurística altera o critério de prioridade da busca ao incorporar informação sobre a posição do objetivo.

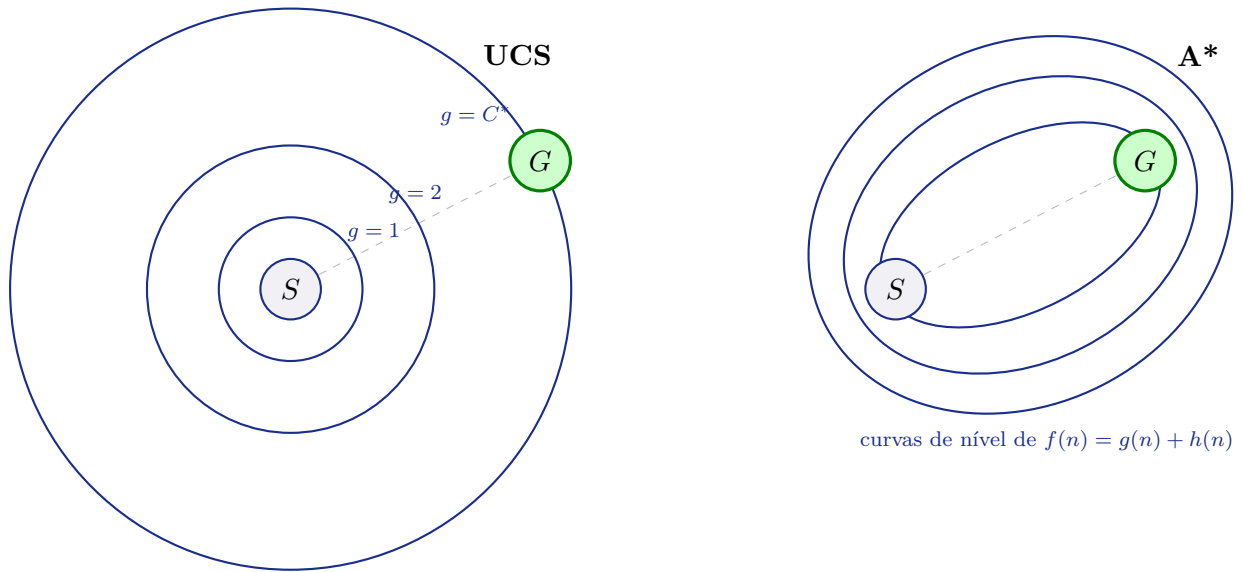


Figura 10: Representação esquemática das curvas de nível da função $f(n) = g(n) + h(n)$ no A*. No caso idealizado em que $g(n)$ é a distância desde S e $h(n)$ é a distância até G , essas curvas são elipses cujos focos são S e G . Valores menores de f correspondem a regiões mais concentradas em torno dos caminhos entre o estado inicial e o objetivo. Em grafos discretos, essas curvas devem ser interpretadas apenas como uma visualização intuitiva do critério de prioridade do A*.

9 Construção de Heurísticas Admissíveis

Saber que o A* é ótimo com heurísticas admissíveis levanta uma questão prática: *como construir heurísticas admissíveis?* A maior parte do esforço envolvido na solução de problemas de busca complexos está exatamente nessa etapa.

9.1 Problemas relaxados

A abordagem mais sistemática para criar heurísticas admissíveis é resolver versões **simplificadas** (ou *relaxadas*) do problema original.

Um **problema relaxado** é obtido a partir do problema original removendo-se algumas restrições sobre as ações. O custo ótimo de um problema relaxado é sempre menor ou igual ao custo ótimo do problema original. Portanto, o custo ótimo do problema relaxado pode ser usado como uma heurística admissível para o problema original.

A intuição é simples. Ao remover restrições, o agente passa a ter mais liberdade de movimento ou de ação. Com mais opções disponíveis, o problema relaxado nunca pode ser mais difícil que o problema original. Assim, se denotarmos por $h^*(n)$ o custo ótimo restante no problema original e por $h_R^*(n)$ o custo ótimo restante em uma versão relaxada, então

$$h_R^*(n) \leq h^*(n).$$

Logo, podemos definir

$$h(n) = h_R^*(n),$$

obtendo uma heurística admissível.

A ideia central para uma forma útil de procurar uma heurística admissível é perguntar: “*Que restrição do problema original eu poderia remover para tornar o problema muito mais fácil de resolver?*”. Se o problema relaxado puder ser resolvido rapidamente, seu custo ótimo pode servir como uma estimativa admissível para o problema original.

A seguir, revisitamos alguns problemas já apresentados nesta nota de aula. Em cada caso, a heurística pode ser entendida como o custo ótimo de uma versão relaxada do problema original.

Exemplo 1: labirinto. No problema de navegação em um labirinto, o agente só pode se deslocar por posições livres; as paredes restringem os caminhos possíveis.

Uma relaxação natural consiste em *remover as paredes*. O agente passa então a poder caminhar diretamente pelo plano, sem precisar contornar obstáculos.

Se movimentos em qualquer direção forem permitidos no problema relaxado, o custo ótimo passa a ser a distância Euclidiana entre a posição atual e o objetivo:

$$h_E(n) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}.$$

Como remover paredes só pode encurtar o caminho, temos

$$h_E(n) \leq h^*(n).$$

Portanto, a distância Euclidiana é admissível.

Se mantivermos a restrição de movimentos apenas horizontais e verticais, mas ainda removermos as paredes, o custo ótimo do problema relaxado será a distância de Manhattan:

$$h_M(n) = |x_2 - x_1| + |y_2 - y_1|.$$

Novamente,

$$h_M(n) \leq h^*(n),$$

e portanto h_M também é admissível.

Exemplo 2: cidades da Romênia. No problema das cidades da Romênia, as ações disponíveis são restritas às estradas existentes entre cidades vizinhas.

Uma relaxação consiste em permitir que o agente viaje *diretamente em linha reta entre quaisquer duas cidades*, mesmo que não exista uma estrada ligando-as.

Nesse problema relaxado, o custo ótimo para ir de uma cidade n até Bucareste é simplesmente sua distância em linha reta até Bucareste. Assim,

$$h(n) = d_{\text{reta}}(n, \text{Bucareste}).$$

Como qualquer rota rodoviária precisa percorrer uma distância pelo menos tão grande quanto a distância em linha reta,

$$d_{\text{reta}}(n, \text{Bucareste}) \leq h^*(n).$$

Portanto, a distância em linha reta é uma heurística admissível quando o custo do problema é a distância percorrida.

A escolha da heurística deve, contudo, ser coerente com a função de custo do problema. Se o objetivo fosse minimizar o *tempo de viagem*, a distância em linha reta, medida em quilômetros, não poderia ser usada diretamente como estimativa de tempo. Nesse caso, se v_{max} é um limite superior para a velocidade em qualquer trecho, podemos definir

$$h(n) = \frac{d_{\text{reta}}(n, \text{Bucareste})}{v_{\text{max}}}.$$

Mesmo no cenário mais favorável, nenhum veículo poderia percorrer a distância em linha reta em menos tempo que esse valor. Logo, essa função fornece um limite inferior para o tempo real restante e é admissível para um problema cujo custo é medido em tempo.

Pergunta 10

Por que a distância em linha reta é uma heurística admissível quando o custo do problema é a distância percorrida, mas não é, por si só, uma heurística para tempo de viagem? Explique por que $d_{\text{reta}}(n, \text{Bucareste})/v_{\text{max}}$ fornece uma heurística admissível para tempo.

Observe a interpretação: não estamos afirmando que o agente realmente pode viajar em linha reta no problema original. Essa ação existe apenas no *problema relaxado*. Seu custo ótimo fornece um limite inferior para o custo do problema real.

Exemplo 3: quebra-cabeça de 8 peças. No quebra-cabeça de 8 peças, o agente só pode mover para o espaço vazio uma peça que esteja adjacente a ele. Essa restrição é responsável por grande parte da dificuldade do problema. Ao removê-la de maneiras diferentes, obtemos problemas relaxados distintos e, conseqüentemente, diferentes heurísticas admissíveis. Considere duas relaxações clássicas, descritas a seguir.

- *Relaxação 1: peças fora do lugar.* Suponha que qualquer peça possa ser movida diretamente para sua posição objetivo em um único passo, independentemente de sua posição atual e da localização do espaço vazio.

Nesse problema relaxado, basta mover uma vez cada peça que estiver fora de sua posição correta. Assim, o custo ótimo é

$$h_1(n) = \text{número de peças fora de suas posições objetivo.}$$

Essa heurística é admissível porque, no problema original, cada peça fora do lugar precisará participar de pelo menos um movimento antes que o estado objetivo seja alcançado. Como o problema relaxado permite movimentos que não existem no problema original, seu custo ótimo não pode exceder o custo real:

$$h_1(n) \leq h^*(n).$$

- *Relaxação 2: distância de Manhattan.* Considere agora uma relaxação menos permissiva. Cada peça continua podendo se mover apenas horizontal ou verticalmente, mas passa a ignorar as demais peças e não precisa estar adjacente ao espaço vazio para se deslocar.

Nesse problema relaxado, cada peça pode seguir independentemente um caminho mínimo até sua posição objetivo. O número mínimo de movimentos necessários para uma peça é sua distância de Manhattan até essa posição. Portanto,

$$h_2(n) = \sum_i d_M(i, \text{posição objetivo de } i).$$

A heurística h_2 também é admissível. No problema original, as interações entre as peças e a necessidade de usar o espaço vazio podem apenas acrescentar movimentos; não podem tornar a solução mais curta que aquela obtida quando essas restrições são removidas. Logo,

$$h_2(n) \leq h^*(n).$$

As duas heurísticas resultam da remoção de restrições do mesmo problema original, mas em graus diferentes. A relaxação associada a h_1 é mais permissiva: qualquer peça fora do lugar pode ser colocada diretamente em sua posição objetivo em um único movimento. Já a relaxação associada a h_2 preserva mais da estrutura original, pois cada peça ainda precisa percorrer horizontal e verticalmente as linhas e colunas que a separam de sua posição objetivo. Por isso, h_1 produz um limite inferior mais fraco, enquanto h_2 tende a fornecer uma estimativa mais próxima de $h^*(n)$. De fato, $h_2(n) \geq h_1(n)$ para todo estado n . Dizemos, portanto, que h_2 é mais informada que h_1 .

Por exemplo, considere uma peça que esteja a três posições de distância de sua posição objetivo. Para h_1 , essa peça contribui apenas com 1, pois está fora do lugar. Para h_2 , ela pode contribuir com 3, refletindo melhor o esforço mínimo necessário para reposicioná-la.

Essa diferença tem impacto direto no A*. Como h_2 fornece estimativas mais próximas do custo real sem superestimá-lo, ela tende a permitir que o algoritmo descarte mais cedo regiões pouco promissoras do espaço de busca.

Experimentos típicos com o quebra-cabeça de 8 peças ilustram esse efeito:

Algoritmo / Heurística	4 passos	8 passos	12 passos
UCS	112	6.300	$3,6 \times 10^6$
A* com h_1 (peças fora do lugar)	13	39	227
A* com h_2 (distância Manhattan)	12	25	73

A tabela mostra a média de nós expandidos para problemas cuja solução ótima possui diferentes comprimentos. Ambas as heurísticas preservam a otimalidade do A*, mas a heurística mais informada h_2 reduz substancialmente o número de expansões.

Pergunta 11

Nos exemplos do labirinto, das cidades da Romênia e do quebra-cabeça de 8 peças, identifique explicitamente qual restrição foi removida para construir o problema relaxado e qual heurística resulta dessa remoção. No caso do quebra-cabeça de 8 peças, explique também por que h_2 é mais informada que h_1 .

Pergunta 12

Compare as heurísticas h_1 (peças fora do lugar) e h_2 (distância Manhattan) para o quebra-cabeça de 8 peças. Qual delas é mais informada? Por que isso implica que A* com h_2 expande no máximo tantos nós quanto A* com h_1 ? Ambas são admissíveis? Justifique.

9.2 Exemplo de projeto de heurística

Até aqui, vimos heurísticas já prontas e mostramos como interpretá-las a partir de problemas relaxados. Vamos agora inverter o processo: dado um novo problema, como podemos projetar uma heurística admissível?

Para isso, considere o problema das panquecas (*Pancake Problem*), um problema clássico de busca combinatória. Diferentemente dos exemplos anteriores, ele não envolve navegação em um espaço físico, mas a reorganização de uma estrutura discreta.

Formulação do problema. Considere uma pilha de P panquecas de tamanhos distintos, rotuladas de 1 a P , da menor para a maior. O objetivo é ordenar a pilha de modo que

$$[1, 2, 3, \dots, P]$$

represente a menor panqueca no topo e a maior no fundo.

A única ação disponível é *virar um prefixo da pilha*. Escolhe-se uma posição k e todas as k panquecas do topo são invertidas. Vamos denotar esta ação $\text{flip}(k)$. O custo da ação $\text{flip}(k)$ é k , isto é, o número de panquecas envolvidas na virada. Por exemplo,

$$[4, 3, 2, 1, 5] \xrightarrow{\text{flip}(4)} [1, 2, 3, 4, 5].$$

As quatro panquecas do topo são invertidas, enquanto a panqueca inferior permanece na mesma posição. A Figura 11 ilustra essa operação.

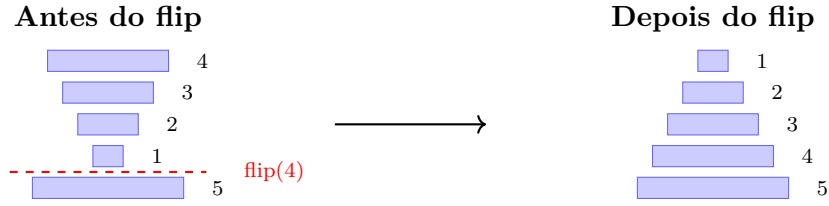


Figura 11: Exemplo de uma ação no problema das panquecas. A operação $\text{flip}(4)$ inverte a ordem das quatro panquecas do topo e tem custo 4.

Como projetar uma heurística? Queremos uma função que seja barata de calcular, mas que forneça alguma informação sobre o custo ainda necessário para ordenar a pilha. Além disso, para preservar a garantia de otimalidade do A^* , queremos que essa função nunca superestime o custo ótimo restante. Uma estratégia é procurar alguma operação que *necessariamente* terá custo elevado em qualquer solução. Como o custo de uma ação $\text{flip}(k)$ é k , panquecas grandes que ainda estão fora de posição fornecem uma pista natural: para posicioná-las corretamente, será necessário realizar alguma virada suficientemente longa. Isso sugere considerar a maior panqueca que ainda não está em sua posição correta.

Uma heurística candidata. Definimos $h(n)$ como o *rótulo da maior panqueca que ainda não está em sua posição correta no estado representado pelo nó n* . Por exemplo, no estado

$$[3, 1, 4, 2, 5]$$

a maior panqueca fora de posição é a de tamanho 4. Portanto,

$$h(n) = 4.$$

Resta verificar se essa estimativa é realmente admissível.

Por que essa heurística é admissível? Seja

$$p = h(n).$$

Assim, p é o rótulo da maior panqueca que ainda está fora de sua posição correta. Como todas as panquecas de rótulo maior que p já estão corretamente posicionadas, elas ocupam as posições

$$p + 1, p + 2, \dots, P.$$

Consequentemente, a panqueca p necessariamente ocupa uma das primeiras p posições. Como ela está fora de posição, sua posição atual é algum valor

$$i < p.$$

Em algum momento de qualquer solução, essa panqueca terá de ser levada à posição p .

Considere a virada que finalmente a coloca nessa posição. Se, imediatamente antes dessa ação, a panqueca p estiver na posição i , uma operação $\text{flip}(k)$ leva a posição i para a posição

$$k - i + 1.$$

Para que o resultado seja a posição p , deve valer

$$k - i + 1 = p.$$

Logo,

$$k = i + p - 1.$$

Como $i \geq 1$,

$$k \geq p.$$

Portanto, qualquer solução precisa conter pelo menos uma ação cujo custo seja

$$k \geq p.$$

Como o custo total da solução é pelo menos o custo dessa ação,

$$h^*(n) \geq p.$$

Mas, por definição,

$$p = h(n).$$

Logo,

$$h(n) \leq h^*(n).$$

Portanto, a heurística é admissível.

Pergunta 13

Considere como heurística alternativa o número de panquecas fora de posição. Essa função também parece fornecer alguma informação sobre a distância até o objetivo. Ela é admissível para a função de custo definida neste problema? Como ela se compara à heurística baseada na maior panqueca fora de posição?

Este exemplo ilustra o processo de projeto de uma heurística. Primeiro, identificamos uma propriedade estrutural do problema: viradas que envolvem muitas panquecas são caras. Em seguida, procuramos uma característica do estado que imponha um custo inevitável a qualquer solução; nesse caso, a maior panqueca fora de posição fornece esse limite inferior. Por fim, mostramos que esse limite nunca supera $h^*(n)$, garantindo a admissibilidade da heurística e permitindo seu uso em algoritmos de busca informada.

9.3 O trade-off entre qualidade da heurística e custo por nó

Existe um equilíbrio delicado no projeto de heurísticas:

- Quanto mais a heurística se aproxima do custo real $h^*(n)$, menos nós o A* expande.
- Porém, heurísticas mais informadas geralmente exigem mais tempo de cálculo por nó.

- No extremo, usar $h(n) = h^*(n)$ evita expandir nós cujo custo total estimado excede o custo ótimo. Ainda podem existir empates entre nós em caminhos ótimos, resolvidos pela regra de desempate; calcular $h^*(n)$, porém, é tão difícil quanto resolver o problema original.

Pergunta 14

Suponha que, em um dado problema, a heurística perfeita $h^*(n)$ é conhecida e computável em tempo constante. Que nós o A* certamente deixaria de expandir? Como empates entre caminhos ótimos podem afetar os nós efetivamente expandidos? O que isso revela sobre o papel da heurística na determinação do comportamento do A*?

A heurística ideal é aquela que fornece a melhor estimativa possível dado o tempo de cálculo disponível. Na prática, heurísticas derivadas de problemas relaxados tendem a ser um bom equilíbrio entre qualidade e eficiência de cálculo.

9.4 Dominância entre heurísticas

Dadas duas heurísticas admissíveis h_a e h_b , dizemos que h_a domina h_b se:

$$h_a(n) \geq h_b(n) \quad \text{para todo } n$$

Em A* em árvore com heurísticas admissíveis, ou em A* em grafo com heurísticas consistentes (ou com reabertura correta de estados), se h_a domina h_b , então A* com h_a expande no máximo tantos nós quanto A* com h_b (e geralmente expande menos). Portanto, nessas condições, se temos múltiplas heurísticas admissíveis disponíveis, devemos preferir aquela que domina as demais.

Uma consequência importante: o máximo de várias heurísticas admissíveis também é admissível:

$$h(n) = \max\{h_1(n), h_2(n), \dots, h_k(n)\}$$

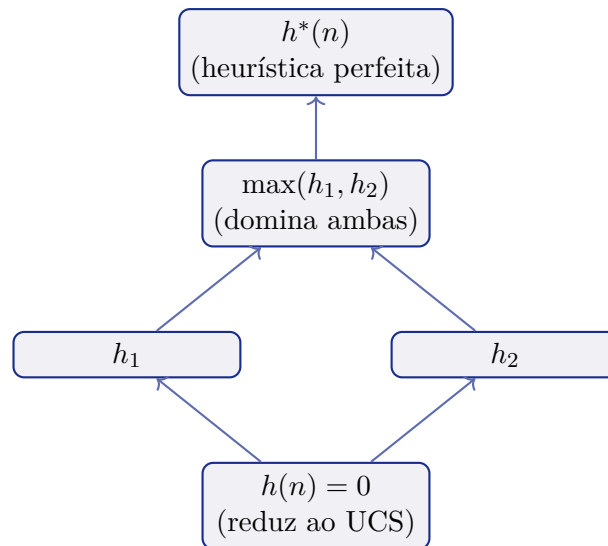
e essa heurística combinada domina todas as heurísticas individuais. Se as heurísticas componentes também forem consistentes, o máximo delas também será consistente; assim, a combinação preserva igualmente a condição necessária para A* em grafo sem reabertura de estados.

Pergunta 15

Considere que você tem duas heurísticas admissíveis h_1 e h_2 para um mesmo problema, e que nenhuma delas domina a outra (ou seja, existem estados onde $h_1 > h_2$ e estados onde $h_2 > h_1$). Como você combinaria as duas para obter uma heurística que domina ambas? Mostre que a combinação resultante é ainda admissível.

O semi-reticulado das heurísticas admissíveis

É útil visualizar o conjunto das heurísticas admissíveis para um problema como uma estrutura hierárquica, ilustrada abaixo:



Na base está a heurística nula $h(n) = 0$, que reduz o A* à UCS. No topo está a heurística perfeita $h^*(n)$, que nunca é conhecida em problemas não-triviais (calculá-la equivale a resolver o problema). Cada nó na hierarquia domina todos os que estão abaixo dele. A operação de máximo permite “subir” nessa estrutura: combinando heurísticas admissíveis pelo máximo, obtemos uma heurística que domina cada uma das componentes. Essa estrutura é chamada de *semi-reticulado* das heurísticas admissíveis.

10 Aplicações do A*

O A* é um dos algoritmos mais utilizados na prática em inteligência artificial e em computação em geral. Algumas de suas aplicações incluem:

- **Videogames:** pathfinding de personagens em mapas de grade ou navmeshes. É o padrão industrial para movimentação de NPCs (*non-player characters*).
- **Sistemas de navegação:** encontrar rotas em mapas rodoviários (aplicativos como Waze e Google Maps utilizam variantes do A*).
- **Robótica:** planejamento de movimento de robôs em espaços contínuos ou discretizados.
- **Planejamento de recursos:** otimização de rotas de entrega, escalonamento de tarefas.
- **Processamento de linguagem natural:** tradução automática, reconhecimento de fala.

A versatilidade do A* vem do fato de que ele pode ser aplicado a qualquer problema formulável como um problema de busca. Uma heurística admissível e razoável é necessária para obter a garantia de otimalidade e, em geral, boa eficiência; heurísticas inadmissíveis também podem ser usadas quando se aceita a possibilidade de soluções subótimas.

11 Resumo Comparativo

A tabela abaixo sumariza as propriedades de pior caso dos algoritmos estudados nesta e nas notas de aula anteriores.

Algoritmo	Função $f(n)$	Compl.?	Ótimo?	Tempo	Espaço
BFS	–	Sim	Sim [†]	$O(b^d)$	$O(b^d)$
DFS	–	Não	Não	$O(b^m)$	$O(bm)$
UCS	$g(n)$	Sim	Sim	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$
Busca Gulosa	$h(n)$	Não	Não	$O(b^m)$	$O(b^m)$
A*	$g(n) + h(n)$	Sim	Sim*	$O(b^{C^*/\varepsilon})$	$O(b^{C^*/\varepsilon})$

[†] Ótimo apenas quando todos os custos de ação são iguais (unitários).

* Com heurística admissível (tree search) ou consistente (graph search).

b : fator de ramificação; d : profundidade da solução ótima; m : profundidade máxima;

C^* : custo da solução ótima; ε : menor custo de ação.

A complexidade $O(b^{C^*/\varepsilon})$ apresentada na tabela refere-se ao *pior caso*, no qual a heurística não fornece nenhuma informação útil, situação equivalente a $h(n) = 0$ para todo n , que reduz o A* à UCS. Quando os custos são unitários, C^*/ε coincide com a profundidade d da solução ótima e obtemos a forma usual $O(b^d)$.

Na prática, o desempenho do A* é melhor descrito pelo conceito de *fator de ramificação efetivo* b^* , definido como o fator de ramificação que uma árvore uniforme precisaria ter para conter exatamente N nós até a profundidade d , onde N é o número de nós efetivamente expandidos pelo A*:

$$N = 1 + b^* + (b^*)^2 + \dots + (b^*)^d.$$

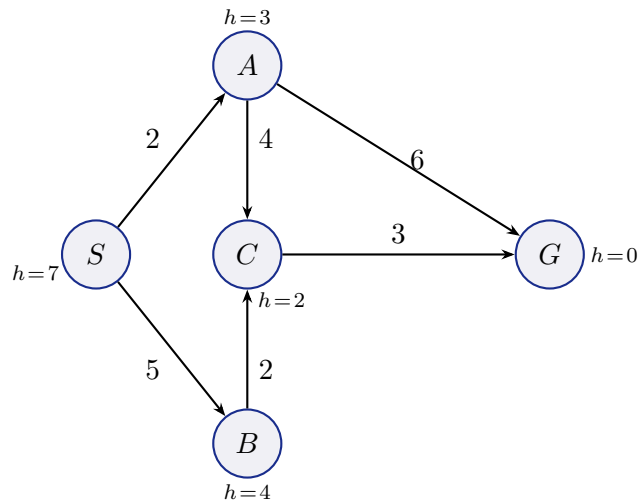
O valor de b^* é obtido resolvendo essa equação numericamente; para árvores grandes com $b^* > 1$, a aproximação $N \approx (b^*)^d$ pode ser útil. Com uma heurística de boa qualidade, $b^* \ll b$, e a diferença é *exponencial* no número de nós expandidos.

Os dados da Seção 9 ilustram isso concretamente: para o quebra-cabeça de 8 peças com solução de comprimento 12, a UCS expande cerca de $3,6 \times 10^6$ nós, enquanto o A* com h_2 (distância Manhattan) expande apenas 73, uma redução de quatro ordens de grandeza que não aparece na análise assintótica de pior caso. O valor b^* pode ser usado empiricamente para avaliar e comparar a qualidade de heurísticas: quanto mais próximo de 1, mais eficiente é a busca.

12 Atividades Intra-Classe

12.1 Busca Gulosa vs. A* em um grafo simples

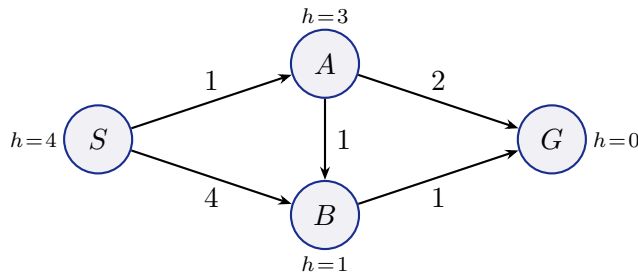
Considere o grafo a seguir, com heurísticas indicadas:



- Execute a Busca Gulosa a partir de S para encontrar um caminho até G . Registre a sequência de nós expandidos e o caminho encontrado.
- Execute o A^* a partir de S para encontrar o caminho de menor custo até G . Registre a sequência de nós expandidos, os valores de $f = g + h$ para cada nó expandido, e o caminho ótimo encontrado.
- Compare os resultados: a Busca Gulosa encontrou a solução ótima? Quantos nós cada algoritmo expandiu?

12.2 Verificação de admissibilidade e consistência

Considere o grafo abaixo com os valores de h indicados:



- Para cada estado, calcule o custo real ótimo $h^*(n)$ até G .
- A heurística h é admissível? Verifique para cada estado.
- A heurística h é consistente? Verifique a desigualdade triangular para cada aresta.
- Execute o A^* com busca em grafo. A solução encontrada é ótima? O resultado concorda com sua análise de consistência?

12.3 Construção de heurísticas por problemas relaxados

Considere o problema de ordenação por inversão de prefixo (problema das panquecas) com 4 panquecas.

- (a) Formule o problema como um problema de busca (S , ACTIONS, T , c , s_0 , GoalTest).
- (b) Proponha uma heurística admissível baseada em um problema relaxado. Justifique a admissibilidade.
- (c) Para o estado $[3, 1, 4, 2]$ (do topo para a base), calcule o valor da sua heurística.
- (d) Existe uma heurística mais informada? Descreva-a e verifique se ela também é admissível.